Computational Complexity

Reading guide and Homework for **Week 05**

1 Reading Guide

- Space complexity: PSPACE and NSPACE, and basic inclusions. §4.1
- The TQBF problem, its PSPACE-completeness and Savitch's Theorem. §4.2
- We hinted at the proof that Sokoban is PSPACE-complete. This is nowhere in the book. A reasonable proof would use the gadgets of:

  Joseph C. Culberson. 1997. *Sokoban is PSPACE-complete.*

  combined with the (much simpler) Turing-machine simulation of an earlier paper (which proves that a variant of Sokoban is PSPACE-hard):

  Dorit Dor and Uri Zwick. 1996. *Sokoban and other motion planning problems.*

  Possible project for the second part of the course: write down a complete proof of the PSPACE-hardness of Sokoban, with the above outline.

There were several details that we covered very quickly, and which **must** be looked at carefully, in order to properly understand what is going on. This kind of familiarity will be necessary to understand what's being said in class.

2 Homework

Skill: *Reason about space.*

1. A *two-player polynomial game* is a family of games. Each game is defined by a polynomial-time Turing machine M, as follows. In the *length-n* version of the game, the game proceeds in $2n$ rounds. Player 1 chooses a string $x_1 \in \{0,1\}^n$, and shows it to Player 2. Then Player 2 chooses a string $x_2 \in \{0,1\}^n$, which can depend on x_1, and shows it to Player 1, *etc*, with Player 1 playing on odd rounds and Player 2 playing on even rounds, with each move allowed to depend on the previous moves. In the end, Player 1 wins if $M(x_1, \ldots, x_n)$ outputs 1, and Player 2 wins otherwise.

   - Show that $n \times n$ checkers is (in the sense that *it can be encoded as*) a two-player polynomial game, if we impose a poly(n) limit on the number of moves.
   - Convince yourself that any reasonable $n \times n$ variant of Chess that one could conceive, is a two-player polynomial game.
   - Why is it reasonable to expect that every single two-player game with a polynomial limit on the number of moves, is a two-player polynomial game?
   - Show that in any such game either Player 1 can always win, or Player 2 can always wing.
   - Show that the problem of deciding, when given 1^n and M, whether Player 1 or Player 2 can always win, is in PSPACE.

2. Suppose you are programming a recursive procedure with the following general outline:

   procedure f(x)
   if x is "simple", then just output $f(x)$ using a simple subroutine
   otherwise, compute $x_1, \ldots, x_k$ somehow based on x
   call f on each x_i, and use the results to obtain $f(x)$

   many algorithms follow the above pattern, often with $k = 1$, but sometimes with larger k. E.g. algorithms that process tree-like data structures, divide and conquer algorithms, *etc*.

   Suppose that the algorithm is guaranteed to never recurse to more than d depth (i.e. f can call f can call f *etc* no more than d times). Suppose that the variables in the body of procedure f use $S = S(n)$ space (here $n = |x|$, clearly $S \geq (1+d)n$ since we need to store x and the x_i at least), and that the time taken to carry out the operations in the main body of the procedure, just the simple operations, i.e. without counting the time of the recursive calls, is $T = T(n)$. How much space in total does a call to $f(x)$ need to use? (as a function of S, d, k) What is the maximum ammount of time that such a procedure can take (as a function of T, d, k)

Skill: *Show PSPACE-completeness.*

3. Exercise 4.2 (of the book)

4. The problem of *reachability in implicitly given graphs* is as follows. We are given as input a sequence 1^n, a Turing machine M and a sequence 1^t. These three inputs define a graph $G(n, M, t)$ over the vertex set $\{0,1\}^n$, as follows: in $G(n, M, t)$, there is an edge from

vertex $x \in \{0,1\}^n$ to vertex $y \in \{0,1\}^n$ if and only if $M(x,y)$ halts and outputs 1 within t steps.

Our problem is then:

$$\mathsf{ImplicitReach} = \{\langle 1^n, M, 1^t \rangle \mid \text{there is a path from } 0^n \text{ to } 1^n \text{ in } G(n,M,t)\}.$$

Show that ImplicitReach is PSPACE-complete under polytime reductions.